

RV32I Processor

Three-stage pipeline



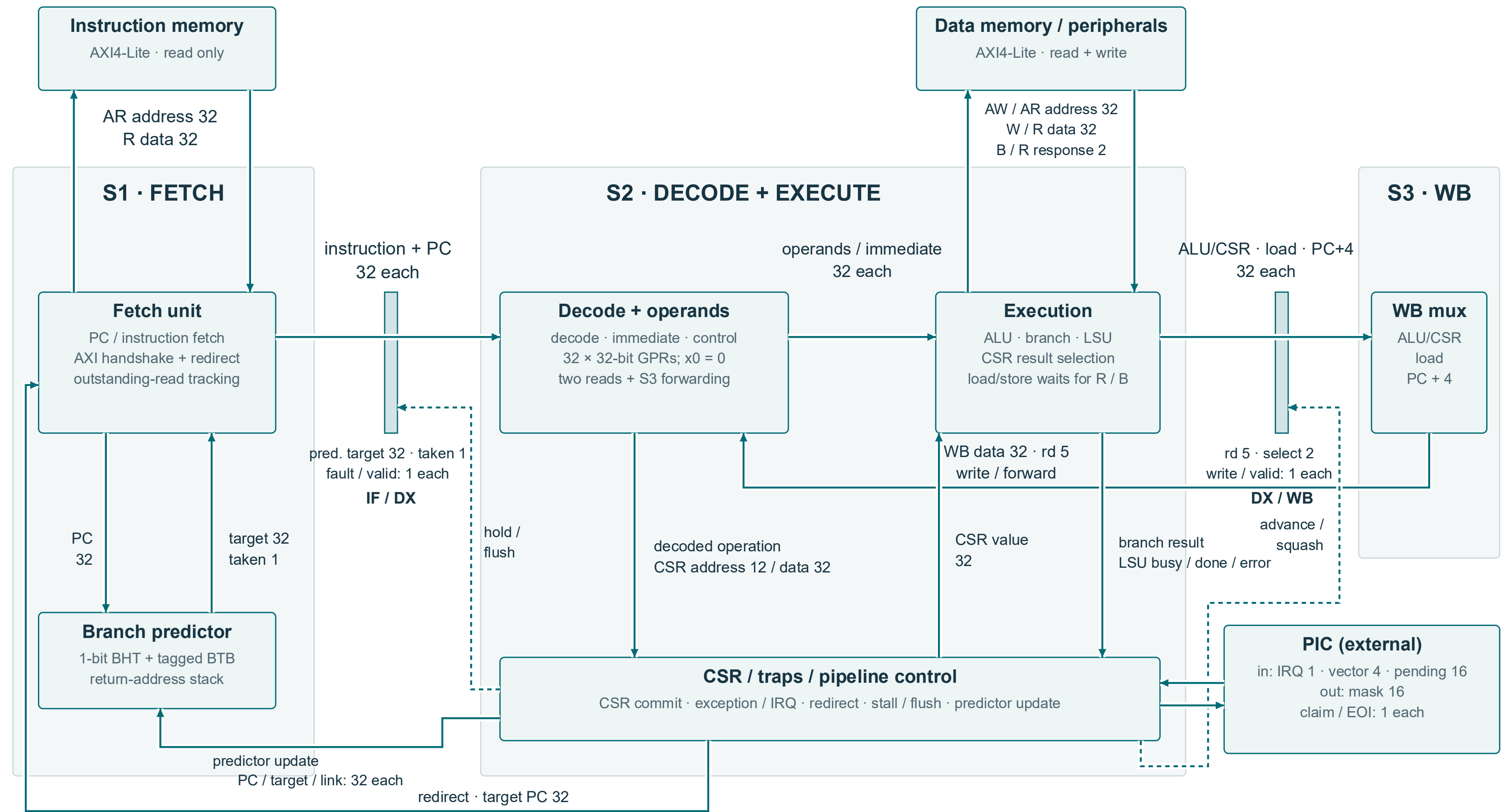
Bunea Cosmin-Andrei
Computer Engineering · year III · academic year 2025–2026
Transilvania University of Braşov

RTL implementation · interfaces · verification

The task: a three-stage RV32I core with two AXI4-Lite ports

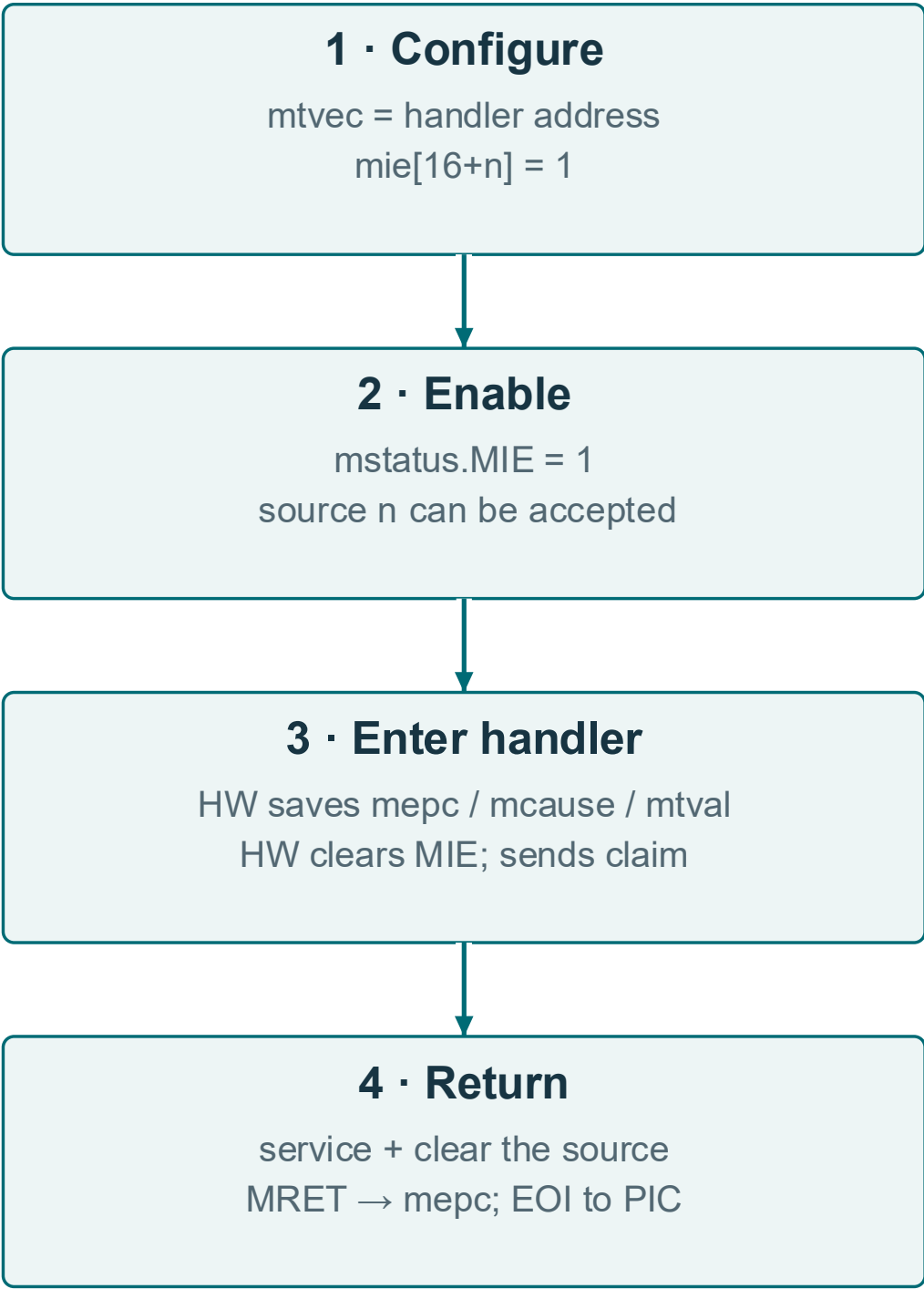
- 01** RV32I set: arithmetic, logic, control transfer, memory
- 02** Three stages: fetch, decode + execute, writeback
- 03** 32×32 -bit register file; x0 hardwired to zero
- 04** Branch prediction with one direction bit
- 05** Exceptions and interrupts at instruction boundaries
- 06** Two AXI4-Lite ports: instruction and data
- 07** CSRs for control, traps and IRQ state
- 08** Centralized control for stall, flush and hazards
- 09** Defined behaviour for AXI latency and errors
- 10** Interrupt link to the PIC

Three stages; the memory access completes inside S2



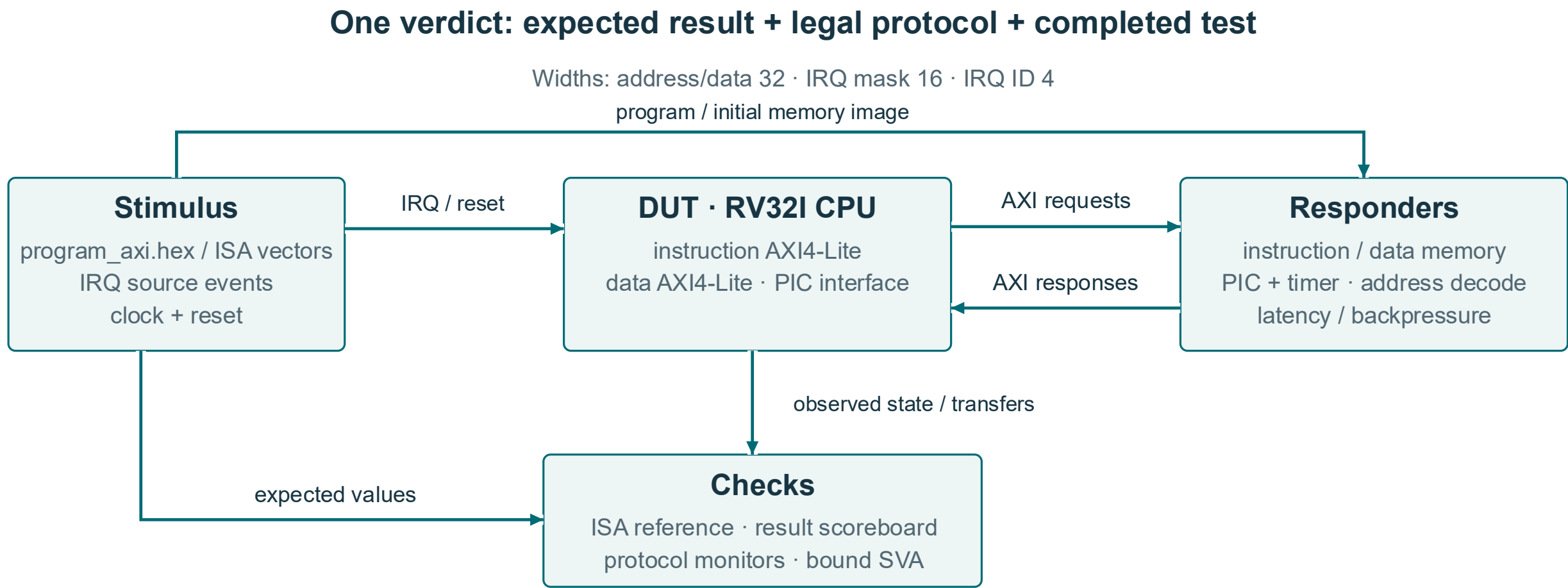
The CSRs link configuration to trap entry and return

Address	Register / group	Role in use
0x300	mstatus	Global MIE; state on entry / return
0x304 / 344	mie / mip	Per-source mask / pending requests
0x305	mtvec	Handler base; direct or vectored
0x341–343	mepc / mcause / mtval	Return PC; cause; fault payload
0x340	mscratch	Context managed by software
0xB00 / B02	mcycle / minstret	Cycles and retired instructions



CPU instructions ↔ hardware state

The functional result and the bus protocol are checked separately



Independent model

49,149 instructions with expected results

Protocol + state

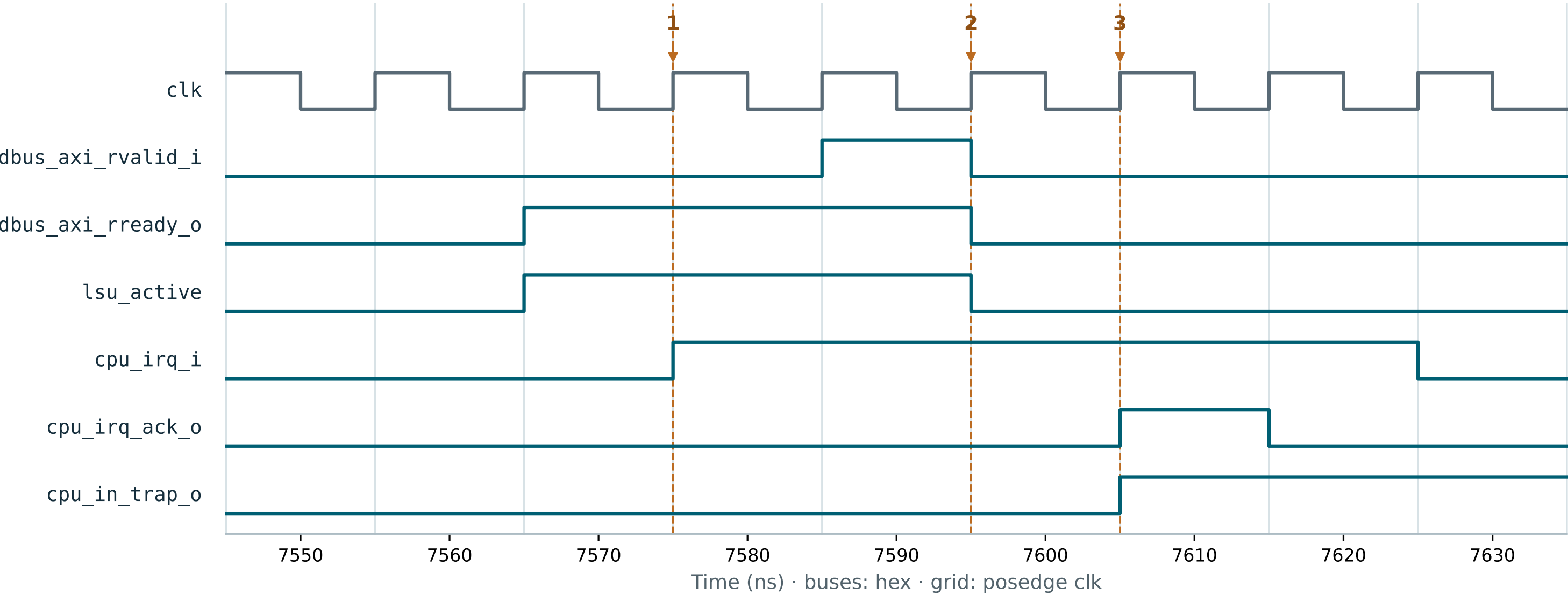
AXI monitors, scoreboard and SVA through bind

Critical cases

IRQ in stall · flush with AXI response · async reset

ModelSim: 39 CPU/PIC runs · Verilator: 17 benches, 53 runs with assertions

The IRQ waits for the load to finish before the claim



1 • New IRQ

LSU busy → claim = 0

2 • R handshake

Load complete → S2 may advance

3 • Claim

Trap entry; one-cycle pulse

The markers follow the same instruction and the same IRQ request.

Result: an integrated core, checked instruction by instruction

WHAT I BUILT

1. RV32I pipeline, predictor and forwarding integrated
2. CSRs, traps and AXI access with explicit rules
3. Execution compared against an independent ISA model

WHAT I LEARNED

- I defined the stall / flush / trap order before integration.
- I separated protocol checking from the architectural effect.
- I used concurrent scenarios to test the boundaries between blocks.

My contribution: design decisions → RTL → integration → verification